

Lekcja 07 - Data contracts

2026-08-17

Data contracts w Pythonie

```
raw payload -> validation -> accepted/rejected -> metrics
```

■ Problem biznesowy

Dashboard pokazuje revenue, statusy i marze.

CF0, ops i PM opieraja decyzje na tych liczbach.

Jesli input jest niejednoznaczny, koszt bledu jest wyzszy niz koszt naprawienia kontraktu na wejsci.

```
nie ma zlego kodu transformacji,  
jest zly input i brak jawnego kontraktu.
```

CZESC I: Data contract i schema drift

- 1) Co to jest data contract i po co istnieje
- 2) Gdzie kontrakt siedzi w pipeline
- 3) Jak wyglada schema drift i jak reagowac
- 4) Jak projektowac reject policy i quality KPIs
- 5) Jak testowac kontrakt

CZESC II: Concurrency w Pythonie

- 6) Threading, multiprocessing, asyncio w DE
- 7) Jak opowiadac o tym na interview

Pipeline bez kontraktu jest jak formularz bez walidacji.

```
kazdy wpis przechodzi,  
wiec blad trafia do raportu
```

Kontrakt to bramka na wejściu.

■ Problem biznesowy

Dashboard pokazuje revenue.

CF0 podejmuje decyzje na tej liczbie.

Jesli dane sa zle, decyzja tez bedzie zla.

Data quality = business risk

■ Skąd biorą się błędy

```
Producer zmienił payload  
Nowy status bez uzgodnienia  
Typ pola zmieniony na string  
Wiecej nulli po nowej wersji API
```

Nie ma złego kodu transformacji. Jest zły input.

■ Contract definition

Data contract opisuje:

```
required fields  
field types  
allowed values  
range constraints  
nullability  
reject policy
```

To jest techniczna umowa producer-consumer.

■ Boundary thinking

Podział odpowiedzialności:

```
Boundary layer  -> walidacja payloadu  
Domain layer   -> model biznesowy  
Business layer  -> metryki i agregacje
```

Każda warstwa ma inny cel.

```
read raw dict  
-> validate with Pydantic  
-> convert to dataclass  
-> compute metrics on accepted only  
-> write accepted/rejected outputs
```

```
record = {  
    "order_id": "1001",  
    "status": " Completed ",  
    "total_amount": "120.50",  
}
```

`dict` jest elastyczny, ale nie daje gwarancji kontraktu.

■ Przykład: payload poprawny

```
{  
  "order_id": "1001",  
  "status": "completed",  
  "total_amount": 120.50,  
  "source": "api"  
}
```

Decyzja:

```
ACCEPT  
-> mapuj do dataclass Order  
-> licz do KPI biznesowych
```

■ Przykład: payload bledny

```
{  
  "status": "done",  
  "total_amount": "120.50",  
  "source": ["api"]  
}
```

Co jest zle:

```
brak order_id  
status poza enum  
total_amount ma zly typ dla strict mode (string zamiast float)  
source ma zly typ (list zamiast str)
```

Decyzja: REJECT + reasons, bez liczenia do revenue.

```
from pydantic import BaseModel, ConfigDict, Field

class OrderPayload(BaseModel):
    model_config = ConfigDict(
        extra="forbid",
        strict=True,
        str_strip_whitespace=True,
    )

    order_id: str
    status: str
    total_amount: float = Field(ge=0)
    source: str = "unknown"
```

Pydantic sprawdza dane wtedy, kiedy naprawdę przychodzi.

```
ALLOWED_STATUSES = {"completed", "cancelled", "pending", "refunded"}

@field_validator("status")
@classmethod
def normalize_status(cls, value: str) -> str:
    normalized = value.strip().lower()
    if normalized not in ALLOWED_STATUSES:
        raise ValueError("status must be one of allowed values")
    return normalized
```

Zysk:

```
" Completed " -> completed (accept)
"DONE"        -> reject
```

■ Why runtime validation

Typy statyczne nie sprawdzają JSON-a z API.

```
mypy pomaga programiscie,  
Pydantic chroni runtime input
```

Potrzebujesz obu, ale do innego celu.

■ Status normalization

Walidator statusu robi dwie rzeczy:

```
normalizuje input (strip + lower)
egzekwuje enum wartosci
```

To ogranicza warianty typu `Completed`, `COMPLETED`, `completed`.

■ Allowed status contract

```
completed  
cancelled  
pending  
refunded
```

Kazdy inny status:

```
reject + reason
```

■ Accept vs reject – szybka tabela

record status	total_amount	decyzja
-----	-----	-----
completed	120.50	ACCEPT
Completed	120.50	ACCEPT (po normalizacji)
pending	0.00	ACCEPT
done	120.50	REJECT (invalid enum)
completed	-10.00	REJECT (range)

Kontrakt ma być deterministyczny: ta sama decyzja dla tego samego payloadu.

■ dataclass in domain

```
@dataclass(frozen=True)
class Order:
    order_id: str
    status: str
    total_amount: float
    source: str
```

To jest rekord, na którym liczysz metryki.

■ Pydantic vs dataclass

```
Pydantic    -> walidacja wejscowa  
Dataclass   -> prosty model domenowy
```

Wzorzec:

```
Pydantic na brzegu, dataclass w srodku
```

■ Required fields gate

Przed walidacja kontraktu zrob szybki check:

```
czy record ma order_id, status, total_amount
```

To daje czytelne "missing field" jako osobna klasa odrzucenia.

■ Accepted / rejected model

```
accepted -> idzie do metryk  
rejected -> idzie do analizy jakosci
```

Nigdy nie licz revenue na rejected.

■ Rejected record shape

Minimalne pola:

```
record
  reason
  validation_stage
  source_system
  ingested_at
  contract_version
```

Im lepsza diagnostyka, tym szybszy incident response.

■ Rejected record – realny wpis

```
{
  "record": {"status": "done", "total_amount": "120.50"},
  "reason": "missing required field: order_id; invalid status enum",
  "validation_stage": "boundary_contract",
  "source_system": "partner_api",
  "ingested_at": "2026-08-27T09:05:10Z",
  "contract_version": "v1"
}
```

To jest dowód do debugowania, nie tylko "błąd w logach".

■ Reject reasons examples

```
missing required field: order_id  
status: must be one of allowed values  
total_amount: should be >= 0  
source: expected string, got list
```

Reason musi być technicznie precyzyjny.

■ Schema drift

Schema drift = zmiana payloadu po stronie producenta.

```
pole znika  
pole zmienia typ  
pojawia sie nowy enum  
zmienia sie semantyka pola
```

■ Drift classification

```
Backward compatible:  
    nowa opcjonalna kolumna  
  
Soft breaking:  
    nowy status bez wsparcia logiki  
  
Hard breaking:  
    brak required field albo zmiana typu
```

Każdy typ wymaga innej reakcji.

■ Reakcja na drift

- 1) wykryj przez walidacje i metryki reject
- 2) sklasyfikuj rodzaj zmiany
- 3) update kontraktu + testow
- 4) komunikacja z ownerem zrodla

Najgorsze: ciche przyjecie nieznanej semantyki.

■ Contract versioning

```
v1 -> status enum A  
v2 -> status enum A + B
```

Dobra praktyka:

```
kazdy record i raport ma contract_version
```

■ Versioning – przykład v1 -> v2

v1:

```
{"order_id":"1001","status":"completed","total_amount":120.50}
```

v2 (additive):

```
{"order_id":"1001","status":"completed","total_amount":120.50,"  
currency":"EUR"}
```

Zasada rolloutu:

```
okres dual-read -> aktualizacja testów -> podniesienie  
contract_version
```

■ Quality KPIs

Monitorujesz minimum:

```
accepted_count  
rejected_count  
reject_rate  
top_reject_reasons  
reject_rate_by_source
```

To sygnały operacyjne, nie "nice to have".

■ Incident trigger

Przykład:

```
reject_rate rosl z 0.8% do 18%
```

To nie jest drobny blad. To jest alert i investigation.

■ Testing strategy

Musisz testować:

```
missing required field  
wrong type  
invalid enum  
boundary values  
status normalization  
schema drift scenario  
happy path
```

■ Boundary values

Przykłady przypadków granicznych:

```
total_amount = 0  
status = " Completed "  
order_id = ""  
total_amount = -1
```

Te przypadki najczęściej uciekają bez testów.

```
walidacja tylko w dokumentacji  
try/except który polyka błędy  
silent cast wszystkiego do str  
brak reject reason  
metryki liczone na surowych dictach
```

To buduje złudzenie stabilności.

- 1) sprawdź top reject reasons
- 2) porównaj contract_version przed/po deploy
- 3) wyizoluj source z najwyższym reject_rate
- 4) odtwórz payload lokalnie
- 5) przygotuj fix i test regresji

W produkcji kontrakt to zwykle osobny mini-modul:

```
src/pipeline/contracts/  
models.py          -> Pydantic input models  
validators.py      -> enum/range/normalization  
routing.py         -> accepted/rejected split  
versioning.py      -> contract_version + changelog helper  
metrics.py         -> reject_rate, top reasons
```

Zysk: testowalnosc i brak mieszania kontraktu z I/O.

Wartosciowe ustawienia dla kontraktu wejścia:

```
extra="forbid"          -> nie przyjmuj nieznanych pol po cichu  
str_strip_whitespace=True -> trim tekstow na wejsci  
validate_default=True   -> waliduj tez wartosci domyslne
```

Wniosek:

```
lepiej fail fast na boundary niz cichy drift w KPI
```

■ Run metadata (must-have)

Kazdy run pipeline powinien zapisac metadane:

```
run_id
started_at, finished_at
source_system
input_records_count
accepted_count, rejected_count
reject_rate
contract_version
code_version (commit SHA)
```

Bez run metadata nie zrobisz porzadnego incident investigation.

■ Quarantine + replay

Rejected records nie trafiaja do kosza.

```
raw payload -> reject -> quarantine storage  
                -> reason + run_id + contract_version
```

Po fixie:

```
replay quarantined records  
porownaj accepted_count before/after
```

To jest standardowy wzorzec odzyskiwania danych po incydencie.

■ Contract tests w CI

Minimalny zestaw testów gate'ujący merge:

```
unit tests: validators i routing  
fixtures: known-good + known-bad payloads  
regression: payloady z poprzednich incydentów  
snapshot: shape quality_summary.json
```

Policy:

```
bez zielonych contract tests -> brak merge do main
```

■ Schema evolution playbook

Zmiana kontraktu powinna mieć jawny rollout:

- 1) Additive change: dodaj pole opcjonalne
- 2) Dual-read period: wspieraj stary i nowy format
- 3) Deprecation notice: komunikat do producenta
- 4) Breaking cutover: podnies `contract_version`

Najgorszy scenariusz: "breaking change bez wersji i bez komunikacji".

Przykład prostych progów operacyjnych:

```
reject_rate <= 1.0%      -> OK  
reject_rate 1.0%-5.0%    -> warning + ticket  
reject_rate > 5.0%       -> incident + stop downstream publish  
missing required field spike -> incident critical
```

Data quality bez progów = brak triggera do działania.

■ Incident case study (timeline)

Przykład realnego dnia on-call:

```
09:05 reject_rate skacze z 0.7% do 14%  
09:08 alert na Slack + ticket P1  
09:15 top reason: missing order_id (source=partner_api)  
09:25 producer potwierdza rollback endpointu  
09:40 replay z quarantine po fixie  
10:05 reject_rate wraca do 0.9%
```

Wniosek: kontrakt + metryki skrocily czas od wykrycia do recovery.

Sytuacja	-> Akcja
-----	-----
Nowe pole opcjonalne	-> accept + log + update modelu
Nowa wartosc enum uzgodnienie	-> tymczasowy reject +
Brak required field	-> reject + incident
Zmiana typu (float -> string)	-> reject + hotfix producera
Nadmiarowe pole krytyczne	-> fail fast (extra=forbid)

Cel: ta sama decyzja niezaleznie od osoby na dyzurze.

■ Payload before/after (drift)

Before (v1):

```
{"order_id":"1001","status":"completed","total_amount":120.5}
```

After (drift):

```
{"status":"done","total_amount":"120.50","currency":"EUR"}
```

Routing:

```
order_id missing    -> reject
status invalid enum -> reject
total_amount string -> reject (strict=True)
currency extra      -> reject (extra=forbid)
```

■ Dashboard minimalny (operacje)

Co pokazujesz codziennie zespołowi:

```
reject_rate trend (24h / 7d)
top_reject_reasons
reject_rate_by_source
contract_version distribution
time_to_recover po incydencie
```

Bez dashboardu drifty wracaja i kazdy incydent zaczyna sie od zera.

Concurrency w Pythonie dla Data Engineering:

```
multithreading  
multiprocessing  
asyncio  
GIL w praktyce
```

Teraz przechodzimy z "poprawnosci" do "wydajnosci bez utraty poprawnosci".

■ Typowy Python w DE: concurrency

Data contract to logika.

Ale w zyciu dochodzi tez wydajnosc:

```
duzo plikow  
duzo payloadow  
wolne I/O
```

■ Co to jest proces?

Proces = uruchomiony program z własną pamięcią i zasobami OS.

```
Python process:  
- osobna pamięć (RAM)  
- osobny interpreter  
- osobny GIL  
- PID w systemie
```

W praktyce:

```
multiprocessing = wiele procesów  
-> prawdziwy paralelizm CPU  
-> ale droższa komunikacja między procesami
```

■ Co to jest watek?

Watek = sciezka wykonania wewnatrz JEDNEGO procesu.

Thread w Python process:

- wspoldzieli pamiec procesu
- wspoldzieli obiekty i zmienne
- wspoldzieli JEDEN GIL

W praktyce:

```
multithreading = wiele watkow  
-> swietne dla I/O (czekanie na API/dysk)  
-> uwaga na race conditions (shared state)
```

■ HTTP w analogii Tiangolo (hamburgery)

Mapowanie backendu HTTP na restauracje:

Klient HTTP	= klient przy kasie
Request /orders	= zamówienie burgera
API server	= restauracja
Handler endpointu	= kelner obsługujący zamówienie
DB / external API	= kuchnia i dostawcy składników
Response JSON	= gotowy burger wydany klientowi

Klucz Tiangolo:

HTTP jest I/O-bound (czekanie na sieć i DB),
więc async/await pozwala obsłużyć wielu klientów naraz,
zamiast blokować jeden request na czas czekania.

■ Sync vs async HTTP (burger line)

SYNC:

kelner bierze 1 zamówienie i czeka przy kuchni aż gotowe
-> kolejni klienci stoja i czekają

ASYNC:

kelner oddaje zamówienie kuchni i idzie do kolejnego klienta
-> obsługuje wiele stolików, wraca gdy kuchnia sygnalizuje gotowe

W FastAPI / httpx AsyncClient to jest dokładnie ten model.

Model mentalny, który wyjaśnia wszystko:

```
Kelnerzy = watki (threading)
    czekaja na stolik, kuchnie, kase
    nie gotuja – CZEKAJA i obsluguja

Kucharze = procesy (multiprocessing)
    kazda kuchnia niezalezna, swoj sprzet
    naprawde rownolegle gotowanie (CPU)

Super-kelner = asyncio
    jeden kelner, 200 stolikow
    nigdy nie czeka bezczynnie – zawsze cos robi
```

■ GIL – szef sali z kluczem do kasy

GIL = tylko jeden wątek naraz wykonuje kod Pythona

Ale podczas I/O (czekanie na dysk/siec) GIL jest ZWOLNIONY:

Kelner czeka na kuchni -> GIL zwolniony
Inny wątek może działać -> threading ma sens dla I/O

Kelner sam gotuje (pure Python CPU) -> GIL trzymany
Inne wątki stoją -> threading NIE pomaga

Rozwiązanie dla CPU: oddzielne kuchnie = `ProcessPoolExecutor`.

■ Multithreading - kiedy

Uzyj dla I/O-bound:

```
czytanie wielu plikow  
requesty do API  
zapis wielu malych outputow  
upload na S3
```

Wzorzec:

```
with ThreadPoolExecutor(max_workers=8) as pool:  
    results = list(pool.map(read_file, paths))
```

■ Multiprocessing - kiedy

Uzyj dla CPU-bound:

```
ciezka normalizacja stringow  
hashing i checksums  
duze lokalne walidacje regex  
transformacje Pandas/NumPy bez Spark
```

Wzorzec:

```
with ProcessPoolExecutor(max_workers=4) as pool:  
    results = list(pool.map(heavy_transform, records))
```

Uwaga: dane sa KOPIOWANE przez pickle – drogie dla duzych rekordow.

■ Asyncio - kiedy

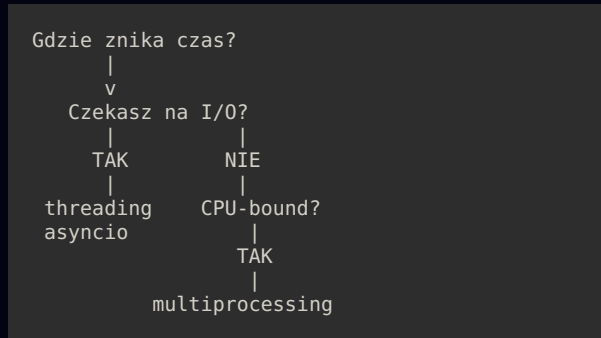
Sens gdy klient i stack są async end-to-end:

```
httpx.AsyncClient  
setki równoległych requestów HTTP  
async-native DB (asyncpg, motor)
```

```
async with httpx.AsyncClient() as client:  
    results = await asyncio.gather(*[fetch(client, url) for url in  
    urls])
```

Nie mieszaj async + sync blokujący – to blokuje całego kelnera.

■ Diagram decyzyjny



malo danych (< 1000 rek.) -> sequential, nie komplikuj.

■ Bezpieczny wzorzec dla kontraktu

- 1) threading w extract (I/O-bound)
- 2) walidacja kontraktu deterministyczna
- 3) kazdy watek ZWRACA wynik (nie dopisuje do shared listy)
- 4) main thread agreguje accepted/rejected
- 5) sort outputu przed zapisem

Klucz: concurrency zmienia kolejnosc, nie decyzje walidacyjna.

```
accepted.append() wewnatrz watku    -> race condition
brak limitu max_workers              -> 1000 watkow dla 1000
plikow
ThreadPoolExecutor dla malych danych -> overhead > zysk
asyncio.run() wewnatrz ThreadPool    -> crash event loop
brak baseline pomiaru                -> moze byc WOLNIEJ
```

Wynik: flaky testy i trudny debug.

■ Rule of thumb

```
I/O-bound (pliki, API, DB) -> threading lub asyncio  
CPU-bound (obliczenia)      -> multiprocessing  
malo danych (< 1000 rek.)   -> sequential  
nie wiesz                   -> zmierz, nie zgaduj
```

Pytanie:

How do you enforce data contracts in Python pipelines?

Odpowiedz:

I validate raw payloads at the pipeline boundary using Pydantic. Accepted records are converted into domain dataclasses. Invalid records are stored with explicit reject reasons. I monitor reject-rate metrics and version the contract over time. For performance I choose threading for I/O-bound stages and multiprocessing for CPU-bound work.

■ Interview follow-up

Pytanie:

What happens when the producer changes schema unexpectedly?

Odpowiedz:

We fail fast for breaking changes, capture rejects with details, compare reject metrics before and after the change, and update contract + tests in a controlled release.

Oddajesz:

```
order_contract.py  
validate_orders.py  
test_order_contract.py  
accepted_orders.jsonl  
rejected_orders.jsonl  
notes.md  
interview_answer.md
```

■ Definition of Done

```
[ ] kontrakt jest w kodzie  
[ ] accepted i rejected sa rozdzielone  
[ ] kazdy reject ma reason  
[ ] testy pokrywaja drift i granice  
[ ] quality metrics sa policzone  
[ ] uzasadniony wybor concurrency model
```

■ Closing

Po tej lekcji umiesz obronic wiarygodnosc pipeline'u.

```
Nie tylko "działa".  
Działa i chroni metryki przed zlym inputem.
```